

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

Лекция 11а

Объекты JavaScript

1. Объекты как ассоциативные массивы
2. Перебор свойств
3. Передача по ссылке
4. Методы объектов
5. Преобразование объектов: `toString` и `valueOf`
6. Создание объектов через `"new"`. Конструктор.
7. Дескрипторы, геттеры и сеттеры свойств

ОБЪЕКТЫ КАК АССОЦИАТИВНЫЕ МАССИВЫ

Ассоциативный массив – структура данных, в которой можно хранить любые данные в формате ключ-значение



`objectName.propertyName`

```
var myCar = new Object();  
myCar.make = "Ford";  
myCar.model = "Mustang";  
myCar.year = 1969;
```

```
myCar["make"] = "Ford";  
myCar["model"] = "Mustang";  
myCar["year"] = 1969;
```

СОЗДАНИЕ ОБЪЕКТОВ

1. `o = new Object();`
2. `o = {};` // пустые фигурные скобки или инициализатор объекта
// со списком свойств=значений
3. Функция конструктор
4. Метод `Object.create`

Операции с объектом

```
var person = {}; // пока пустой
```

объект.свойство

```
// при присвоении свойства в объекте автоматически создаётся "ячейка"
```

```
// с именем "name" и в него записывается содержимое 'Вася'
```

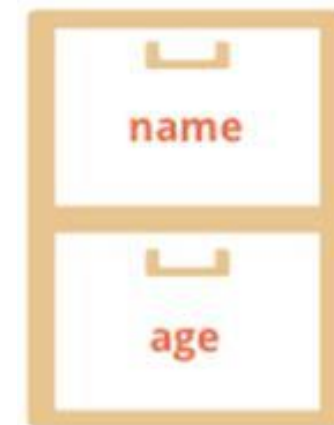
```
person.name = 'Вася';
```

```
// запишем ещё одно свойство:
```

```
//с именем 'age' и значением 25
```

```
person.age = 25;
```

```
alert( person.name + ': ' + person.age ); // "Вася: 25"
```



можно обратиться к любому свойству объекта, даже если его нет. Ошибки не будет.

если свойство не существует, то вернется undefined:

```
var person = {};
```

```
alert( person.lalala ); // undefined, нет свойства с ключом lalala
```

```
var person = { name: "Василий"};
```

```
alert( person.lalala === undefined ); // true, свойства нет
```

```
alert( person.name === undefined ); // false, свойство есть.
```

ДОСТУП ЧЕРЕЗ КВАДРАТНЫЕ СКОБКИ

```
var myObj = new Object(),  
    str = "myString",  
    rand = Math.random(),  
    obj = new Object();  
myObj.type          = "Dot syntax";  
myObj["date created"] = "String with space";  
myObj[str]          = "String value";  
myObj[rand]         = "Random Number";  
myObj[obj]          = "Object";  
myObj[""]           = "Even an empty string";  
  
console.log(myObj);
```

```
[object Object] {  
  : "Even an empty string",  
  0.9527493129767106: "Random Number",  
  [object Object]: "Object",  
  date created: "String with space",  
  myString: "String value",  
  type: "Dot syntax"  
}
```

ДОСТУП ЧЕРЕЗ КВАДРАТНЫЕ СКОБКИ

объект['свойство']:

```
var person = { };
```

```
person['name'] = 'Вася'; // то же что и person.name = 'Вася'
```

```
person['любимый стиль музыки'] = 'Джаз';
```

```
person.любимый стиль музыки = 'Джаз'; // ошибка
```

```
alert(person['любимый стиль музыки']);
```


Доступ к свойству через переменную

к свойству, имя которого хранится в переменной, можно обратиться с помощью квадратных скобок:

```
var person = {};  
person.age = 25;  
var key = 'age';  
alert( person[key] ); // выведет person['age']
```

```
//используется , когда название свойства введено  
//посетителем и записано в переменную
```

Доступ к свойству через переменную

```
var myCar = new Object();
```

```
var propertyName = "make";
```

```
myCar[propertyName] = "Ford";
```

```
propertyName = "model";
```

```
myCar[propertyName] = "Mustang";
```

```
console.log(myCar);
```

```
myCar.color; // undefined
```

```
[object Object] {  
  make: "Ford",  
  model: "Mustang"  
}
```

Удаление свойства

```
var person = {};
```

```
person.name = 'Вася';
```

```
person.age = 25;
```

```
delete person.age;
```



проверить, есть ли в объекте свойство с определенным ключом

1) оператор: "in".

синтаксис: "prop" in obj, причем имя свойства – в виде строки

```
if ("name" in person) {  
  alert( "Свойство name существует!" );  
}
```

2) сравнение значения с undefined

```
var obj = {};  
obj.test = undefined;  
// добавили свойство со значением undefined
```

```
// проверим наличие свойств  
//test и заведомо  
//отсутствующего blabla  
alert( obj.test === undefined ); // true  
alert( obj.blabla === undefined );// true
```

оператор in гарантирует
правильный результат:

```
alert( "test" in obj ); // true  
alert( "blabla" in obj ); // false
```

Объявление со свойствами

Объект можно заполнить значениями при создании, указав их в фигурных скобках (литеральный синтаксис):

{ ключ1: значение1, ключ2: значение2, ... }

```
var menuSetup = { width: 300, height: 200, title: "Menu"};
```

// то же самое, что:

```
var menuSetup = {};
```

```
menuSetup.width = 300;
```

```
menuSetup.height = 200;
```

```
menuSetup.title = 'Menu';
```

Названия свойств можно перечислять как в кавычках, так и без, если они удовлетворяют ограничениям для имён переменных:

```
var menuSetup = {  
    width: 300,  
    'height': 200,  
    "мама мыла раму": true  
};
```

В качестве значения можно тут же указать и другой объект:

```
var user = {  
  name: "Таня",  
  age: 25,  
  size: {  
    top: 90,  
    middle: 60,  
    bottom: 90  
  }  
}  
  
alert(user.name); // "Таня"  
alert(user.size.top); // 90
```


Компактное представление объектов

```
var user = { name: "Vasya", age: 25};
```

При создании множества объектов одного и того же вида (с одинаковыми полями) интерпретатор выносит описание полей в отдельную структуру. А сам объект остаётся в виде непрерывной области памяти с данными.

{name: "Вася", age: 25}	Вася 25
{name: "Петя", age: 22}	Петя 22
{name: "Маша", age: 19}	Маша 19

```
<структура: string name, number age>
```

ОБЪЕКТЫ: ПЕРЕБОР СВОЙСТВ

- циклы `for...in` – метод перебирает все перечисляемые свойства объекта и его цепочку прототипов
- `Object.keys(o)` – метод возвращает массив со всеми собственными (те, что в цепочке прототипов, не войдут в массив) именами перечисляемых свойств объекта `o`.
- `Object.getOwnPropertyNames(o)` – метод возвращает массив содержащий все имена своих свойств (перечисляемых и неперечисляемых) объекта `o`.

ОБЪЕКТЫ: ПЕРЕБОР СВОЙСТВ

цикл по свойствам `for..in` последовательно перебирает свойства объекта, имя каждого свойства будет записано в `key` и вызвано тело цикла

```
for (key in obj) {  
    /* ... делать что-то с obj[key] ... */  
}
```

Вспомогательную переменную `key` (или другое название) можно объявить прямо в цикле:

```
for (var key in menu) { // ...}
```

```
var menu = {  
    width: 300,  
    height: 200,  
    title: "Menu"  
};
```

```
for (var key in menu) {  
    // этот код будет вызван для каждого свойства объекта  
    // ..и выведет имя свойства и его значение  
    alert( "Ключ: " + key + " значение: " + menu[key] );  
}
```

Количество свойств в объекте

```
var menu = {  
    width: 300,  
    height: 200,  
    title: "Menu "  
};  
  
var counter = 0;  
for (var key in menu) {  
    counter++;  
}  
  
alert( "Всего свойств: " + counter );
```

`Object.keys(menu).length`

```
function isEmpty(obj) {  
    for (var key in obj) {  
        return false;  
    }  
    return true;  
}  
//нет свойств в объекте
```

Порядок перебора свойств

```
var codes = {  
  // телефонные коды в формате "код страны": "название"  
  "7": "Россия",  
  "41": "Швейцария",  
  // ..,  
  "1": "США"  
};
```

если имя свойства – число или числовая строка, то все современные браузеры сортируют такие свойства в целях внутренней оптимизации

если имя свойства – нечисловая строка, то такие ключи всегда перебираются в том же порядке, в каком присваивались

```
var user = {  
    name: "Вася",  
    surname: "Петров "  
};  
user.age = 25;  
// порядок перебора соответствует порядку присвоения свойства  
for (var prop in user) {  
    console.log ( prop );    // name, surname, age  
}
```

```
var codes = {  
    // телефонные коды в формате "код страны": "название "  
    "7": "Россия",  
    "41": "Швейцария",  
    "1": "США "  
};  
for (var code in codes)  
    console.log ( code ); // 1, 7, 41
```



```
var codes = {  
    "+7": "Россия",  
    "+41": "Швейцария",  
    "+1": "США "  
};  
  
for (var code in codes) {  
    var value = codes[code];  
    code = +code;  
  
    // ..если нам нужно именно число, преобразуем: "+7" -> 7  
    console.log ( code + ": " + value ); // 7, 41, 1 во всех браузерах  
}
```

```
"use strict";  
  
var salaries = { "Вася": 100, "Петя": 300, "Даша": 250};  
  
var sum = 0;  
  
for (var name in salaries) {  
    sum += salaries[name];  
}  
  
alert( sum );
```

```
"use strict";
```

```
var salaries = {  
    "Вася": 100,  
    "Петя": 300,  
    "Даша": 250  
};
```

```
var max = 0; var maxName = "";
```

```
for (var name in salaries) {  
    if (max < salaries[name]) {  
        max = salaries[name];  
        maxName = name;    //сотр. с наибольшей зарплатой  
    }  
}
```

```
alert( maxName || "нет сотрудников" );
```

получает объект и умножает все численные свойства на 2

```
var menu = { width: 200, height: 300, title: "My menu"};
function isNumeric(n) {
    return !isNaN(parseFloat(n)) && isFinite(n);
}
function multiplyNumeric(obj) {
    for (var key in obj) {
        if (isNumeric(obj[key])) {
            obj[key] *= 2;
        }
    }
}

multiplyNumeric(menu);

alert( "menu width=" + menu.width + " height=" + menu.height + " title=" +
menu.title );
```

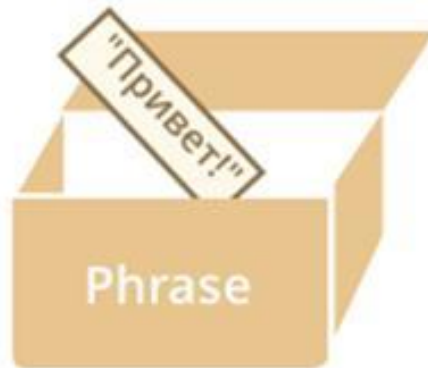
Объекты: передача по ссылке

Обычные значения: строки, числа, булевы значения, null/undefined при присваивании переменных копируются целиком («по значению»)

две полностью независимые переменные:

```
var message = "Привет";
```

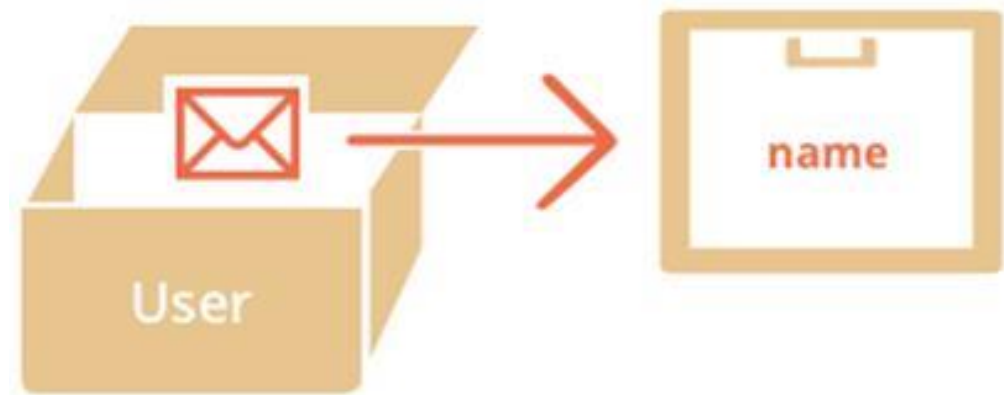
```
var phrase = message;
```



В переменной, которой присвоен объект, хранится не сам объект, а «адрес его места в памяти», иными словами – «ссылка» на него.

переменная, которой присвоен объект:

```
var user = {  
  name: "Вася"  
};
```



При копировании переменной с объектом – копируется эта ссылка, а объект по-прежнему остается в единственном экземпляре.

```
var user = { name: "Вася" }; // в переменной - ссылка
```

```
var admin = user; // скопировали ссылку
```

// две переменные, в которых находятся ссылки на один и тот же объект:



Так как объект всего один, то изменения через любую переменную видны в других переменных:

```
var user = { name: 'Вася' };
```

```
var admin = user; //копия ключа, но сейф по-прежнему один
```

```
admin.name = 'Петя'; // поменяли данные через admin
```

```
alert(user.name); // 'Петя', изменения видны в user
```


Клонирование объектов

```
var user = { name: "Вася", age: 30};  
var clone = {}; // новый пустой объект  
// скопируем в него все свойства user  
for (var key in user) {  
    clone[key] = user[key];  
}
```

```
// теперь clone - полностью независимая копия  
clone.name = "Петя"; // поменяли данные в clone  
alert( user.name ); // по-прежнему "Вася"
```

Object.assign(dest, [src1, src2, src3...])

Методы объектов

Свойства-функции называют «методами» объектов.

```
var user = {  
  name: 'Василий',  
  // метод  
  sayHi: function() {  
    alert( 'Привет!' );  
  }  
};  
  
// Вызов  
user.sayHi();
```

методы объектов можно добавлять и удалять в любой момент, в том числе и явным присваиванием:

```
var user = {  
  name: 'Василий'  
};  
user.sayHi = function() {  
  // присвоили метод после создания объекта  
  alert('Привет!');  
};  
// Вызов метода:  
user.sayHi();
```

Доступ к объекту через this

Для доступа к текущему объекту из метода используется ключевое слово this

```
var user = {  
  name: 'Василий',  
  sayHi: function() {  
    alert( this.name );  
  }  
};  
user.sayHi(); // sayHi в контексте user
```

```
var user = {  
  name: 'Василий',  
  sayHi: function() {  
    alert( user.name ); // приведёт к ошибке  
  }  
};  
var admin = user;  
user = null;  
admin.sayHi();  
// !внутри sayHi обращение по старому имени, ошибка!
```

Через this метод может не только обратиться к любому свойству объекта, но и передать куда-то ссылку на сам объект целиком:

```
var user = {  
  name: 'Василий',  
  sayHi: function() {  
    showName(this); // передать текущий объект в showName  }  
};
```

```
function showName(namedObj) {  
  alert( namedObj.name );  
}
```

```
user.sayHi(); // Василий
```

Значение `this` называется *контекстом вызова* и будет определено в момент вызова функции.

```
function sayHi() { alert( this.firstName );}
```

Если одну и ту же функцию запускать в контексте разных объектов, она будет получать разный `this`:

```
var user = { firstName: "Вася" };
```

```
var admin = { firstName: "Админ" };
```

```
function func() { alert( this.firstName );}
```

```
user.f = func;
```

```
admin.g = func; // this равен объекту перед точкой:
```

```
user.f(); // Вася
```

```
admin.g(); // Админ
```

```
admin['g'](); // Админ (не важно, доступ к объекту через точку или квадратные скобки)
```

Значение this при вызове без контекста

ситуация возникает при ошибке в разработке.

//this получает значение window, глобального объекта:

```
function func() {  
    alert( this );    // выведет [object Window] или [object global]  
}  
func();
```

```
function func() {  
    "use strict";  
    alert( this );    // выведет undefined (кроме IE9-)  
}  
func();
```


ССЫЛОЧНЫЙ ТИП

потеря контекста:

```
var user = {  
  name: "Вася",  
  hi: function() { alert(this.name); },  
  bye: function() { alert("Пока"); }  
};  
  
user.hi(); // Вася (простой вызов работает)  
// а теперь вызовем user.hi или user.bye в зависимости от имени  
(user.name == "Вася" ? user.hi : user.bye)(); // undefined
```

`obj.method()` состоит из двух независимых операций: точка `.` – получение свойства и скобки `()` – его вызов (предполагается, что это функция).

точка возвращает не функцию, а значение специального «ссылочного» типа `Reference Type`:

base – как раз объект,

name – имя свойства,

strict – вспомогательный флаг для передачи `use strict`.

Скобки `()` получают из `base` значение свойства `name` и вызывают в контексте `base`.

Другие операторы получают из `base` значение свойства `name` и используют, а остальные компоненты игнорируют.

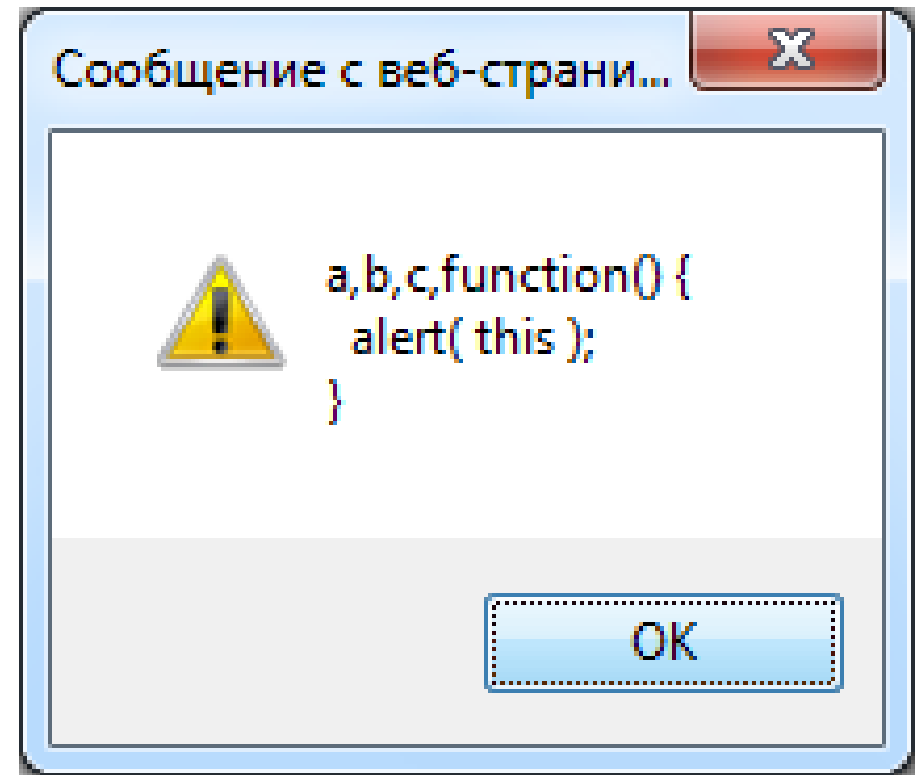
Каким будет результат?

```
var arr = ["a", "b", "c" ];
```

//Вызов в контексте массива

```
arr.push(function() {  
    alert( this );  
})
```

```
arr[3]();
```



Вызов `arr[3]()` – это обращение к методу объекта `obj[method]()`, в роли `obj` выступает `arr`, а в роли метода: `3`.

Поэтому, как это бывает при вызове функции как метода, функция `arr[3]` получит `this = arr` и выведет массив.

Каким будет результат?

```
var obj = {  
  go: function() { alert(this) }  
}  
(obj.go)()
```

Ошибка возникла из-за того, что после объявления obj пропущена точка с запятой. JavaScript игнорирует перевод строки перед скобкой (obj.go)() и читает этот код как:

```
var obj = { go:... }(obj.go)()
```

Интерпретатор попытается вычислить это выражение, которое обозначает вызов объекта { go: ... } как функции с аргументом (obj.go). При этом, естественно, возникнет ошибка.

Каким будет вывод?

```
var user = { firstName: "Василий", export: this};  
alert( user.export.firstName );
```

Ответ: undefined

Объявление объекта само по себе не влияет на `this`. Никаких функций, которые могли бы повлиять на контекст, здесь нет.

Так как код находится вообще вне любых функций, то `this` в нём равен `window` (в браузере так всегда для кода вне функций, вне зависимости от `use strict`).

Получается, что в строке (*) мы имеем `export: window`, так что далее `alert(user.export.firstName)` выводит свойство `window.firstName`, которое не определено.

```
var name = "";  
var user = { name: "Василий",  
export: function() { return this; }  
};  
alert( user.export().name );
```

Ответ: Василий

Вызов `user.export()` использует `this`, который равен объекту до точки, то есть внутри `user.export()` строка `return this` возвращает объект `user`.

В итоге выводится свойство `name` объекта `user`, равное `"Василий"`.

Преобразование объектов: toString и valueOf

объект преобразуется в примитив:

- Строковое преобразование – если объект выводится через alert(obj).
- Численное преобразование – при арифметических операциях, сравнении с примитивом.
- Логическое преобразование – при if(obj) и других логических операциях.

ЛОГИЧЕСКОЕ ПРЕОБРАЗОВАНИЕ

Любой объект в логическом контексте – true, даже если это пустой массив [] или объект {}.

```
if ({} && []) {  
    alert( "Все объекты - true!" ); // alert сработает  
}
```


Строковое преобразование

```
var user = {  
  firstName: 'Василий'  
};
```

```
console.log( user );
```

//выведет [object Object] - стандартное строковое

//представление пользовательского объекта

```
[object Object] {  
  firstName: "Василий"  
}
```

Если в объекте присутствует метод toString, который возвращает примитив, то он используется для преобразования.

```
var user = { firstName: 'Василий',  
  
  toString: function() {  
    return 'Пользователь ' + this.firstName;  
  }  
};  
  
alert( user ); // Пользователь Василий
```

//Результатом toString может быть любой примитив

Метод toString не обязан возвращать именно строку.

«строковое преобразование», а не «преобразование к строке»

```
var obj = {  
  toString: function() {  
    return 123;  
  }  
};  
  
alert( obj +1); // 124
```

Все объекты, включая встроенные, имеют свои реализации метода toString, например:

```
alert( [1, 2] ); // toString для массивов выводит список элементов "1,2 "
```

```
alert( new Date ); // toString для дат выводит дату в виде строки
```

```
alert( function() {} ); // toString для функции выводит её код
```

Численное преобразование

используется метод `valueOf`, а если его нет – то `toString`:

```
var room = { number: 777,  
              valueOf: function() { return this.number; },  
              toString: function() { return this.number; } };  
  
alert( +room );           // 777, вызвался valueOf  
  
delete room.valueOf; // valueOf удалён  
  
alert( +room );           // 777, вызвался toString
```

Метод `valueOf` обязан возвращать примитивное значение, иначе его результат будет проигнорирован. При этом – не обязательно числовое.

У большинства объектов нет `valueOf` (по спецификации есть, но ничего не делает, просто возвращает сам объект (не-примитивное значение!)), а потому игнорируется)

Исключением является объект Date, который поддерживает оба типа преобразований:

```
alert( new Date() ); // toString: Дата в виде читаемой строки
```

```
alert( +new Date() );
```

```
// valueOf: кол-во миллисекунд, прошедших с 01.01.1970
```

После преобразования объекта в примитив при помощи toString или valueOf, примитив может быть преобразован во что-то другое.

операция ==

```
var obj = {
```

```
  valueOf: function() {
```

```
    return 1;  };
```

```
alert( obj == true ); // true
```

```
var obj = { valueOf: function() {  
    return 1; } };  
alert( obj + "test" );  
// 1test
```

для разности объектов:

```
var a = { valueOf: function() {  
    return "1"; } };  
var b = { valueOf: function() {  
    return "2"; } };  
alert( a + b );  
// "12"  
alert( a - b );  
// "1" - "2" = -1
```



```
alert( ['x'] == 'x' ); //??
```

//результат true

Если с одной стороны – объект, а с другой – нет,
то сначала приводится объект.

У массивов нет `valueOf`, поэтому вызывается `toString`, который возвращает список элементов через запятую.

В данном случае, элемент только один – он и возвращается. Так что `['x']` становится `'x'`. Получилось `'x' == 'x'`, верно.

По той же причине верны равенства:

```
alert( ['x', 'y'] == 'x,y' ); // true
```

```
alert( [] == '' ); // true
```

Какими будут результаты alert?

```
var foo = { toString: function() { return 'foo'; },  
            valueOf: function() { return 2; } };
```

```
alert( foo );
```

```
alert( foo + 1 );
```

```
alert( foo + "3" );
```

Первый alert(foo)

Возвращает строковое представление объекта, используя toString, т.е. "foo".

Второй alert(foo + 1)

Оператор '+' преобразует объект к примитиву, используя valueOf, так что результат: 3.

Третий alert(foo + " 3 ")

То же самое, что и предыдущий случай, объект превращается в примитив 2. Затем происходит сложение 2 + '3'. Оператор '+' при сложении чего-либо со строкой приводит и второй операнд к строке, а затем применяет конкатенацию, так что результат – строка "23".

`new Date(0) - 0 = 0 // (1)`

`new Array(1)[0] + "" = "undefined" // (2)`

`({})[0] = undefined // (3)`

`[1] + 1 =`

`[1,2] + [3,4] = "11" // (4)`

`[] + null + 1 = "1,23,4" // (5)`

`[[0]][0][0] = "null1" // (6)`

`({} + {}) = 0 // (7)`

`"[object Object][object Object]" // (8)`

`alert( [1,[5],2][1] );`

5

КОНСТРУКТОР

Конструктором становится любая функция, вызванная через new.

функции, задуманные как конструкторы, называют с большой буквы:

Animal, а не animal

```
function Animal(name) {  
  this.name = name;  
  this.canWalk = true;  
}
```

```
var animal = new Animal("ёжик");
```

```
function Animal(name) {  
  // this = {};  
  // в this пишем свойства, методы  
  this.name = name;  
  this.canWalk = true;  
  // return this;}  
}
```

технически, любая функция может быть использована как конструктор.

действия функции Конструктора:

1. Создаётся новый пустой объект.
2. Ключевое слово `this` получает ссылку на этот объект.
3. Функция выполняется. Как правило, модифицирует `this`, добавляет методы, свойства.
4. Возвращается `this`.

В результате вызова `new Animal("ёжик");` получаем такой объект:

```
animal = { name: "ёжик", canWalk: true }
```

Иногда функцию-конструктор объявляют и тут же используют:

```
var animal = new function() {  
    this.name = "Васька";  
    this.canWalk = true;  
};
```

//то же

```
var animal = { name: "Васька", canWalk: true};
```

// простая запись не подходит, когда при создании свойств объекта нужны более сложные вычисления

ПРАВИЛА ОБРАБОТКИ RETURN

Как правило, конструкторы ничего не возвращают. Их задача – записать всё, что нужно, в `this`, который автоматически станет результатом.

Но если явный вызов `return` всё же есть, то применяется простое правило:

При вызове `return` с объектом, будет возвращён он, а не `this`.

При вызове `return` с примитивным значением, оно будет отброшено.

возврат объекта:

```
function BigAnimal() {  
    this.name = "Мышь";  
    return { name: "Годзилла" }; // <-- возвратим объект  
}  
  
alert( new BigAnimal().name );  
// Годзилла, получили объект вместо this
```



```
function BigAnimal() {  
    this.name = "Мышь";  
    return "Годзилла"; // <-- возвратим примитив  
}
```

```
alert( new BigAnimal().name );  
// Мышь, получили this (а Годзилла пропал)
```

СОЗДАНИЕ МЕТОДОВ В КОНСТРУКТОРЕ

`new User(name)` создает объект с заданным значением свойства `name` и методом `sayHi`:

```
function User(name) {  
  this.name = name;  
  this.sayHi = function() { alert( "Моё имя: " + this.name ); };  
}
```

```
var ivan = new User("Иван");  
ivan.sayHi(); // Моё имя: Иван
```

```
/*ivan = {  name: "Иван",  sayHi: функция}*/
```

Локальные переменные

В функции-конструкторе бывает удобно объявить вспомогательные локальные переменные и вложенные функции, которые будут видны только внутри:

```
function User(firstName, lastName) {  
    // вспомогательная переменная  
    var phrase = "Привет";  
    // вспомогательная вложенная функция  
    function getFullName() {  
        return firstName + " " + lastName;    }  
    this.sayHi = function() {  
        alert( phrase + ", " + getFullName() ); // использование  
    };}  
var vasya = new User("Вася", "Петров");  
vasya.sayHi(); // Привет, Вася Петров
```

Возможны ли такие функции A и B в примере ниже, что соответствующие объекты a,b равны:

```
function A() { ... }
```

```
function B() { ... }
```

```
var a = new A;
```

```
var b = new B;
```

```
alert( a == b ); // true
```

Да, возможны. Они должны возвращать одинаковый объект.

При этом если функция возвращает объект, то `this` не используется.

Например, они могут вернуть один и тот же объект `obj`, определённый снаружи:

```
var obj = {};  
function A() { return obj; }  
function B() { return obj; }  
var a = new A;  
var b = new B;  
alert( a == b ); // true
```

функция-конструктор Calculator, которая создает объект с тремя методами:

```
function Calculator() {  
    this.read = function() {  
        this.a = +prompt('a?', 0);  
        this.b = +prompt('b?', 0);  
    };  
    this.sum = function() {  
        return this.a + this.b;  
    };  
    this.mul = function() {  
        return this.a * this.b;  
    };  
}  
var calculator = new Calculator();  
calculator.read();  
alert( "Сумма=" + calculator.sum() );  
alert( "Произведение=" + calculator.mul() );
```

Дескрипторы, геттеры и сеттеры свойств

Основной метод для управления свойствами – `Object.defineProperty`.
позволяет объявить свойство объекта и тонко настроить его особые аспекты, которые никак иначе не изменить.

Синтаксис:

```
Object.defineProperty(obj, prop, descriptor)
```

Аргументы:

`obj` – Объект, в котором объявляется свойство.

`prop` – Имя свойства, которое нужно объявить или модифицировать.

`descriptor` – Дескриптор – объект, который описывает поведение свойства.

В нём могут быть следующие поля:

`value` – значение свойства, по умолчанию `undefined`

`writable` – значение свойства можно менять, если `true`. По умолчанию `false`.

`configurable` – если `true`, то свойство можно удалять, а также менять его в дальнейшем при помощи новых вызовов `defineProperty`. По умолчанию `false`.

`enumerable` – если `true`, то свойство просматривается в цикле `for..in` и методе `Object.keys()`. По умолчанию `false`.

`get` – функция, которая возвращает значение свойства. По умолчанию `undefined`.

`set` – функция, которая записывает значение свойства. По умолчанию `undefined`.

запрещено одновременно указывать значение `value`, `writable` и функции `get/set`

ОБЫЧНОЕ СВОЙСТВО

Два таких вызова работают одинаково:

```
var user = {};
```

// 1. простое присваивание

```
user.name = "Вася";
```

// 2. указание значения через дескриптор

```
Object.defineProperty(user, "name", { value: "Вася", configurable: true,  
writable: true, enumerable: true });
```

Оба вызова выше добавляют в объект user обычное (удаляемое, изменяемое, перечисляемое) свойство.

СВОЙСТВО-КОНСТАНТА

Для того, чтобы сделать свойство неизменяемым, изменим его флаги writable и configurable:

```
"use strict";
```

```
var user = {};
```

```
Object.defineProperty(user, "name", { value: "Вася",
```

```
  writable: false, // запретить присвоение "user.name="
```

```
  configurable: false // запретить удаление "delete user.name"
```

```
});
```

```
// Теперь попытаемся изменить это свойство.
```

```
// в strict mode присвоение "user.name=" вызовет ошибку
```

```
user.name = "Петя";
```

без use strict операция записи «молча» не сработает. Лишь если установлен режим use strict, то дополнительно сгенерируется ошибка.

СВОЙСТВО, СКРЫТОЕ ДЛЯ FOR...IN

Встроенный метод `toString`, как и большинство встроенных методов, не участвует в цикле `for..in`. Это удобно, так как обычно такое свойство является «служебным».

К сожалению, свойство `toString`, объявленное обычным способом, будет видно в цикле `for..in`, например:

```
var user = { name: "Вася",  
toString: function() { return this.name; } };  
for(var key in user)  
alert(key); // name, toString
```

Object.defineProperty может исключить toString из списка итерации, поставив ему флаг enumerable: false. По стандарту, у встроенного toString этот флаг уже стоит.

```
var user = { name: "Вася",  
  toString: function() { return this.name; };
```

```
// помечаем toString как не подлежащий перебору в for..in
```

```
Object.defineProperty(user, "toString", {enumerable: false});
```

```
for(var key in user)  alert(key); // name
```

вызов defineProperty не перезаписал свойство, а просто модифицировал настройки у существующего toString.

СВОЙСТВО-ФУНКЦИЯ

Дескриптор позволяет задать свойство, которое работает как функция, указав эту функцию в `get`.

Например, у объекта `user` есть обычные свойства: имя `firstName` и фамилия `surname`.

Создадим свойство `fullName` – функцию:

```
var user = { firstName: "Вася", surname: "Петров"}  
Object.defineProperty(user, "fullName", {  
  get: function() { return this.firstName + ' ' + this.surname; }  
});  
  
alert(user.fullName); // Вася Петров
```

Также можно указать функцию, которая используется для записи значения, при помощи дескриптора set.

добавим возможность присвоения user.fullName к примеру выше:

```
var user = { firstName: "Вася", surname: "Петров"}  
Object.defineProperty(user, "fullName", {  
  get: function() { return this.firstName + ' ' + this.surname; },  
  set: function(value) {  
    var split = value.split(' ');  
    this.firstName = split[0];  
    this.surname = split[1];  }  
});  
user.fullName = "Петя Иванов"; //set  
alert( user.firstName ); // Петя  
alert( user.surname ); // Иванов
```

****15 5 УКАЗАНИЕ GET/SET В ЛИТЕРАЛАХ**

Если мы создаём объект при помощи синтаксиса { ... }, то задать свойства-функции можно прямо в его определении.

Для этого используется особый синтаксис: get свойство или set свойство.

ниже объявлен геттер-сеттер fullName:

```
var user = { firstName: "Вася", surname: "Петров",  
  get fullName() { return this.firstName + ' ' + this.surname; },  
  set fullName(value) { var split = value.split(' '); this.firstName = split[0];  
    this.surname = split[1]; }  
};
```

```
alert( user.fullName ); // Вася Петров (из геттера)
```

```
user.fullName = "Петя Иванов";
```

```
alert( user.firstName ); // Петя (установил сеттер)
```

```
alert( user.surname ); // Иванов (установил сеттер)
```

УКАЗАНИЕ GET/SET В ЛИТЕРАЛАХ

```
let user = {  
  name: "John",  
  surname: "Smith",  
  get fullName() {  
    return `${this.name} ${this.surname}`;  
  },  
  set fullName(value) {  
    [this.name, this.surname] = value.split(" ");  
  }  
};  
// set fullName запустится с данным значением  
user.fullName = "Alice Cooper";  
  
alert(user.name); // Alice  
alert(user.surname); // Cooper
```


defineProperty позволяет начать с обычных свойств, а потом заменить их на функции, реализующие более сложную логику

```
function User(name, age) { this.name = name; this.age = age;}  
var pete = new User("Петя", 25);          alert( pete.age ); // 25
```

//изменим функцию

```
function User(name, birthday) { this.name = name; this.birthday = birthday;  
// age будет высчитывать возраст по birthday
```

```
Object.defineProperty(this, "age", {  get: function() {  
    var todayYear = new Date().getFullYear();  
    return todayYear - this.birthday.getFullYear();  }  });}
```

```
var pete = new User("Петя", new Date(1987, 6, 1));  
alert( pete.birthday ); // и дата рождения доступна  
alert( pete.age );      // и возраст
```

ДРУГИЕ МЕТОДЫ РАБОТЫ СО СВОЙСТВАМИ

`Object.defineProperty(obj, descriptors)` – позволяет объявить несколько свойств сразу:

```
var user = {}
```

```
Object.defineProperty(user, {
```

```
  firstName: {  value: "Петя"  },
```

```
  surname: {  value: "Иванов"  },
```

```
  fullName: { get: function() {return this.firstName+ ' ' +this.surname; } } });
```

```
alert( user.fullName ); // Петя Иванов
```

`Object.keys(obj)`, `Object.getOwnPropertyNames(obj)` – возвращают массив – список свойств объекта.

`Object.keys` возвращает только enumerable-свойства.

`Object.getOwnPropertyNames` – возвращает все

```
var obj = { a: 1, b: 2, internal: 3};  
Object.defineProperty(obj, "internal", { enumerable: false});  
alert( Object.keys(obj) ); // a,b  
alert( Object.getOwnPropertyNames(obj) ); // a, b , internal
```

Object.getOwnPropertyDescriptor(obj, prop) – возвращает дескриптор для свойства obj[prop]. Полученный дескриптор можно изменить и использовать defineProperty для сохранения изменений:

```
var obj = { test: 5};  
var descriptor = Object.getOwnPropertyDescriptor(obj, 'test');  
console.log(descriptor);
```

```
[object Object] {  
  configurable: true,  
  enumerable: true,  
  value: 5,  
  writable: true  
}
```

```
// заменим value на геттер, для этого нужно убрать value/writable  
// и поставить get
```

```
delete descriptor.value;  
delete descriptor.writable;  
descriptor.get = function() {  
  alert( " Hi :) " );};
```

```
// поставим новое свойство вместо старого  
delete obj.test;
```

```
Object.defineProperty(obj, 'test', descriptor);  
obj.test; // Hi :)
```

```
[object Object] {  
  configurable: true,  
  enumerable: true,  
  get: function() {  
    alert( "Hi :) " );}  
}
```

используются очень редко:

`Object.preventExtensions(obj)` – Запрещает добавление свойств в объект.

`Object.seal(obj)` – Запрещает добавление и удаление свойств, все текущие свойства делает `configurable: false`.

`Object.freeze(obj)` – Запрещает добавление, удаление и изменение свойств, все текущие свойства делает `configurable: false`, `writable: false`.

`Object.isExtensible(obj)` – Возвращает `false`, если добавление свойств объекта было запрещено вызовом метода `Object.preventExtensions`.

`Object.isSealed(obj)` – Возвращает `true`, если добавление и удаление свойств объекта запрещено, и все текущие свойства являются `configurable: false`.

`Object.isFrozen(obj)` – Возвращает `true`, если добавление, удаление и изменение свойств объекта запрещено, и все текущие свойства являются `configurable: false`, `writable: false`.